

# Write a Linux SPI Device Driver With Ease

This article describes how to write a Linux device driver for a SPI flash memory. However, many parts of the article are general in nature and therefore can be used as a reference while writing drivers for other SPI devices.

The Serial Peripheral Interface (SPI) Bus is a synchronous serial data link standard that operates in full duplex mode. Devices communicate in master/slave mode, where the master device initiates the data frame. Multiple slave devices are allowed with individual slave select (chip select) lines. The SPI is used to talk to a variety of peripherals such as sensors, control devices, flash memories, LCD displays, touchscreens, etc.

In Linux, flash memories are referred to as MTD devices (Memory Technology Devices). In this article, terms like an SPI device, SPI slave device, MTD device and SPI flash are used interchangeably.

## Hardware architecture

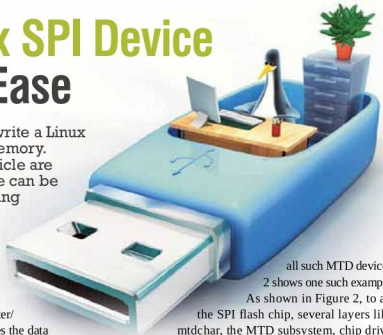
The SPI devices like flash memories, touch-screens and some authentication devices are usually not connected to the CPU directly. Instead, another chip like the PCH (Platform Controller Hub) mediates between the CPU and the SPI device. The PCH is a family of Intel microchips. It is the successor to the previous Intel Hub Architecture, which used the northbridge and the southbridge instead, and first appeared in the Intel 5 Series.

The PCH controls certain data paths and support functions used in conjunction with Intel CPUs. The I/O functions are reassigned between this new central hub and the CPU in comparison to the previous architecture, wherein some northbridge functions like the memory controller and PCI-e lanes were integrated into the CPU while the PCH took over the remaining functions in addition to the traditional roles of the southbridge.

Figure 1 shows a typical hardware connection. The CPU talks to the PCH through the PCI interface. The PCH contains a SPI controller that talks to the flash chip through the SPI interface.

## Driver software architecture

Linux uses a layered architecture to provide access to all MTD devices including NAND-flashes, NOR-flashes, SPI-flashes and



all such MTD devices. Figure 2 shows one such example.

As shown in Figure 2, to access the SPI flash chip, several layers like `mtddchar`, the MTD subsystem, chip driver, SPI framework and SPI controller are involved. Although it may look complex initially, the layered design makes it simple and possible to access any kind of MTD device in any manner. For example, if an MTD device is to be accessed as a block device rather than a character device, then the `mtddchar` layer can be replaced by the Linux VFS, Block I/O and blockdev layers. The blockdev layer provides a flash translation layer that emulates the MTD device as a block device. Therefore, it is possible to use disk-based file systems like `ext2` and `FAT` on the MTD device. Similarly, if an SPI flash chip is replaced by a CFI flash chip, then the chip driver layer is changed to an appropriate chip driver (like `cfi_cmdset_0001.c`). In the same manner, if a Winbond SPI flash is replaced by a Micron SPI flash, then the SPI controller layer should be changed. Therefore, the layered architecture provides reusable components that make it easy to access any kind of MTD device. The only layer that is never changed is the MTD sub-system layer, which is responsible for providing a unified and uniform layer that enables a seamless combination of low-level MTD chip drivers with higher-level interfaces.

Each layer has an assigned job. The chip driver has the responsibility of implementing the chip-specific protocol. Therefore, there are different chip drivers for SPI flashes and CFI flashes because each of them follows a different protocol. This protocol consists of command sets and opcodes understood by the flash chip. The SPI framework is a common layer for all SPI devices. This layer has the responsibility of mediating between the chip driver and the controller driver as you will see later in this article. The SPI controller layer is responsible for actual communication to the SPI device. Therefore, for every unique SPI device, there has to be a SPI controller driver. So let's focus on such SPI controller drivers for SPI flash memory.